

AWiR – ćwiczenie 6

Temat:

Logowanie z wykorzystaniem Spring Security OAuth2 Client (Google).

Cel ćwiczenia:

Rozszerz aplikację z Ćwiczenia 5 (Security) o logowanie federacyjne przez Google:

- dodaj **OAuth2 Login** (Google) obok dotychczasowego logowania formularzowego,
- zezwól użytkownikom przeglądarkowym logować się przyciskiem „Zaloguj przez Google”,
- pozostaw ****REST /api/**** tylko na **HTTP Basic** (bez sesji),
- pokaż dane profilu zalogowanego użytkownika (imię, e-mail, avatar).

Niezbędne narzędzia:

1. DE: STS 4 / IntelliJ / VS Code
2. Maven 3.9+
3. JDK 21
4. Spring Boot 3.3.x
5. Konto Google i dostęp do **Google Cloud Console** (utworzenie OAuth 2.0 Client ID)

Dotychczasowy projekt zawierał:

- Masz już projekt z ćw. 4-5:
 - warstwa **MVC** (/users/**) i **REST** (/api/**),
 - Security: **REST = Basic, stateless; MVC = formLogin + remember-me**,
 - własna strona /login i logika rozdzielona na dwa łańcuchy zabezpieczeń.

W tym ćwiczeniu dodaj:

- zależność **spring-boot-starter-oauth2-client** w pom.xml,
- konfigurację klienta **Google** (client-id, client-secret, redirect URI),
- w SecurityConfig włączyć **oauth2Login()** dla warstwy MVC (obok formLogin),
- przycisk „Zaloguj przez Google” na stronie login.html,
- endpoint / widok z **danymi profilu** (/me),

Zmiany w strukturze projektu (pogrubione)

```
src/main/java/edu/zut/awir/  
├─ AwirApplication.java  
├─ config/  
│   └─ SecurityConfig.java # modyfikujemy  
├─ web/  
│   └─ mvc/  
│       └─ UserController.java
```

```

└─┬─ MeController.java          # NOWE: endpoint z danymi profilu (MVC)
   │ rest/
   │   ├── UserRestController.java
   │   └── RestExceptionHandler.java
└─ service/

```

```

src/main/resources/
└─ application.properties      # NOWE/zmienione (OAuth2)
   └─ templates/
      ├── login.html           # modyfikujemy - przycisk Google
      └── me.html              # NOWE - prezentacja profilu

```

1. Rozszerzenie zależności w pliku pom.xml

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>

```

2. Rejestracja aplikacji w Google

(<https://developers.google.com/identity/protocols/OpenIDConnect>)

- Wejdź do **Google Cloud Console** → „APIs & Services” → **Credentials**.
- **Create Credentials** → **OAuth client ID** → Application type: **Web application**.
- Ustaw **Authorized redirect URI**:
 - `http://localhost:8080/login/oauth2/code/google`
- Zapisz **Client ID** i **Client Secret**.

3. Rozbudowa konfiguracji aplikacji (application.properties)

```

spring.security.oauth2.client.registration.google.client-id=YOUR_GOOGLE_CLIENT_ID
spring.security.oauth2.client.registration.google.client-secret=YOUR_GOOGLE_CLIENT_SECRET
spring.security.oauth2.client.registration.google.scope=openid,profile,email

# Google provider
spring.security.oauth2.client.provider.google.user-name-attribute=sub

```

4. SecurityConfig – włączenie OAuth2 dla MVC

src/main/java/edu/zut/awir/config/SecurityConfig.java

```

@Bean
SecurityFilterChain mvc(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf.ignoringRequestMatchers("/h2/**"))
        .authorizeHttpRequests(reg -> reg
            .requestMatchers("/", "/login", "/oauth2/**", "/css/**", "/js/**",
                "/images/**",
                "/swagger-ui/**", "/v3/api-docs/**", "/h2/**").permitAll()
            .requestMatchers("/users/**", "/me").hasAnyRole("USER", "ADMIN")
            .anyRequest().authenticated()
        )
        // 1) formularz (jak w ćw. 5)
        .formLogin(form -> form
            .loginPage("/login")

```

```

        .loginProcessingUrl("/login")
        .defaultSuccessUrl("/users/list", true)
        .failureUrl("/login?error")
        .permitAll()
    )
    // 2) OAuth2 Login (Google)
    .oauth2Login(oauth -> oauth
        .loginPage("/login")
        .defaultSuccessUrl("/users/list", true)
    )
    .rememberMe(rm -> rm
        .rememberMeParameter("remember-me")
        .key("awir-remember-me-secret-key")
        .tokenValiditySeconds(60 * 60 * 24 * 14)
    )
    .logout(logout -> logout
        .logoutUrl("/logout")
        .logoutSuccessUrl("/login?logout")
        .deleteCookies("JSESSIONID", "remember-me")
        .permitAll()
    )
    .headers(h -> h.frameOptions(f -> f.disable())); // H2
    return http.build();
}

```

5. Przycisk „Zaloguj przez Google” na login.html

src/main/resources/templates/login.html

```

<p style="margin-top:1rem;">
    <a href="/oauth2/authorization/google" style="display:inline-
    block;padding:.5rem .75rem;border:1px solid #ddd;border-radius:6px;text-
    decoration:none;">
        Zaloguj przez Google
    </a>
</p>

```

6. Wyświetlenie profilu - /me

src/main/java/edu/zut/awir/web/mvc/MeController.java

```

package edu.zut.awir.web.mvc;

import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.security.oauth2.core.oidc.user.OidcUser;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class MeController {

    @GetMapping("/me")
    public String me(@AuthenticationPrincipal OidcUser user, Model model) {
        if (user != null) {
            model.addAttribute("name", user.getFullName());
            model.addAttribute("email", user.getEmail());
            model.addAttribute("picture", user.getPicture());
        }
        return "me"; // templates/me.html
    }
}

```

src/main/resources/templates/me.html

```
<!DOCTYPE html>
<html lang="pl" xmlns:th="http://www.thymeleaf.org">
  <head>
    <meta charset="UTF-8" />
    <title>Mój profil</title>
  </head>
  <body>
    <h2>Mój profil</h2>
    <div th:if="{name}">
      
      <p><b>Imię i nazwisko:</b> <span th:text="{name}"></span></p>
      <p><b>E-mail:</b> <span th:text="{email}"></span></p>
    </div>
    <div th:if="{name} == null">
      <p>Brak danych profilu (nie zalogowano przez Google).</p>
    </div>
  </body>
</html>
```

7. Testowanie

- Otwórz /login i kliknij „Zaloguj przez Google”. Po udanym logowaniu trafisz na /users/list.
- Wejdź na /me – powinny być widoczne dane profilu z Google.
- REST /api/** nadal wymaga **Basic Auth** i nie korzysta z sesji/OAuth.

8. Zadania do wykonania

- Dodaj zależność spring-boot-starter-oauth2-client i konfigurację spring.security.oauth2.client w application.yml.
- Skonfiguruj **Google OAuth** (Client ID/Secret, redirect URI) w Google Cloud Console.
- Włącz oauth2Login() w łańcuchu MVC (obok formLogin) i dodaj przycisk na login.html.
- Dodaj stronę /me z danymi profilu (imię, e-mail, avatar).
- Upewnij się, że **REST /api/** nadal wymaga **Basic Auth** (stateless, csrf off).
- Wykonaj testy logowania z poziomu formularza